CS 432 — Database Management Systems

Module A: B+ Tree Indexing & ACID Transaction Engine

CallHub Campus Directory System

GitHub Repository	`https://github.com/Sanjay18255/Assignment_3_Module_A.git`
Video Demonstration	`https://drive.google.com/file/d/1XeLz5PsOKl9Xd62wlZ2ybQh1Ctb_u35x/view?usp=sharing`

Team Contributions

Name	Roll Number	Contribution
Sanjay	23110144	B+ Tree implementation, Table layer, schema validation, test_transactions, report
Thushar	24110393	Transaction engine, WAL logger, crash recovery, ACID tests, report

Contents

1 System Overview **2**
1.1 Architecture . . . 2
1.2 Key Metrics . . . 2

2 B+ Tree Index — Correctness of Operations **3**
2.1 Core Operations and Complexity . . . 3
2.2 Schema Validation . . . 3
2.3 Consistency Verification . . . 4

3 ACID Transactions — Implementation & Failure Handling **4**
3.1 Atomicity . . . 4
3.2 Consistency . . . 4
3.3 Isolation . . . 5
3.4 Durability — Write-Ahead Logging . . . 6

4 Crash Recovery — REDO / UNDO Protocol **6**
4.1 Recovery Algorithm . . . 6
4.2 Identifying Uncommitted Transactions . . . 6
4.3 Crash Simulation and Recovery Test . . . 7
4.4 Robustness — Missing Tables During Recovery . . . 7

5 Multi-User Conflict Handling **7**
5.1 Concurrent Inserts — No Key Collision . . . 7
5.2 Race Condition on Same Primary Key . . . 7
5.3 Stress Concurrency — 50 Sequential Transactions . . . 7

6 Experiments Performed **8**
6.1 Experiment Suite . . . 8
6.2 B+ Tree Performance Benchmarks . . . 8

7 Observations and Limitations **9**
7.1 Key Observations . . . 9
7.2 Limitations . . . 10

8 Conclusion **10**

1. System Overview

CallHub is a campus-wide directory system that stores and manages member profiles, department affiliations, roles, labs, hostels, audit logs, and login histories. Module A implements the entire database engine from scratch — no external DBMS is used. The system is built on three pillars: a custom B+ Tree for indexed storage, a relational table abstraction with schema validation, and a full ACID transaction engine backed by Write-Ahead Logging (WAL).

1.1 Architecture

Table 1: System Architecture

Layer	**Component**	**File**	**Responsibility**
Storage Engine	B+ Tree	bplustree.py	Ordered index, range queries, $O(\log n)$ ops
Table Layer	Table	table.py	Schema enforcement, CRUD, search_key abstraction
Database Layer	DatabaseManager	db_manager.py	Multi-table DDL, transaction gateway
Transaction Layer	Transaction + WALLogger	transaction.py	ACID, rollback, FK checks
Recovery	crash_recovery()	transaction.py	REDO committed, UNDO uncommitted on restart
Tests	acid_tests, test_transactions	`*.py`	Atomicity, Isolation, Durability, stress

1.2 Key Metrics

Table 2: System Summary

Metric	**Value**
Tables Modelled	14
B+ Tree Order	8 (default)
Transaction Operations	INSERT, UPDATE, DELETE
WAL Entries per Txn	≥ 3 (BEGIN + ops + COMMIT/ROLLBACK)
Test Scenarios	10

2. B+ Tree Index — Correctness of Operations

All records are stored in a custom B+ Tree implementation (`bplustree.py`). The tree is the sole storage backend; there is no separate data file. Every table has a designated `search_key` column, and all operations route through the tree for both indexed lookups and full scans.

2.1 Core Operations and Complexity

Table 3: B+ Tree Operations

Operation	Method	Complexity	Correctness Mechanism
Insert	`insert()`	$O(\log n)$	Duplicate check; leaf-split propagation maintains sorted order
Point Lookup	`search()`	$O(\log n)$	Traverses internal nodes to leaf; returns exact match or None
Range Query	`range_query()`	$O(\log n + k)$	Linked leaf list scan from lower to upper bound
Update	`update()`	$O(\log n)$	Locates record by key, overwrites value in-place
Delete	`delete()`	$O(\log n)$	Removes from leaf; borrow/merge to maintain order invariant
Full Scan	`get_all()`	$O(n)$	Traverses entire leaf-linked list left to right
Count/Height	`count()`, `height()`	$O(n)/O(\log n)$	Recursive tree traversal

2.2 Schema Validation

Before any insert or update reaches the B+ Tree, `Table.validate_record()` enforces the schema. Validation checks that every declared column is present, that the value matches the declared Python type (with numeric coercion where safe), and that no numeric field carries a negative value. A failed validation raises an error that propagates to the transaction layer, triggering an automatic rollback.

Listing 1: Schema Validation — table.py

```
def validate_record(self, record):
    for col, expected_type in self.schema.items():
        if col not in record:
            return False, f"Missing required field: '{col}'"
        actual_type = TYPE_MAP.get(expected_type, expected_type)
        if not isinstance(val, actual_type):
            actual_type(val)  # coercion attempt; raises ValueError
    # Negative value guard
    for col, val in record.items():
        if isinstance(val, (int, float)) and val < 0:
            return False, f"Field '{col}' cannot be negative"
    return True, None
```

2.3 Consistency Verification

`check_consistency()` cross-validates the B+ Tree against itself: every key returned by `get_all()` is re-queried via `get()`. If any key is unreachable or if the stored record's `search_key` field does not match the tree key, an issue is recorded. This function runs at the end of every test suite to confirm the index has not been corrupted by any operation.

3. ACID Transactions — Implementation & Failure Handling

Every data modification goes through a `Transaction` object (`transaction.py`). The transaction layer provides full ACID guarantees without any external database engine.

3.1 Atomicity

All operations within a transaction are tracked in an in-memory `ops` list as `(op, table, key, before, after)` tuples. If any operation raises an exception — whether a schema violation, an FK error, or a simulated crash — `rollback()` iterates the list in reverse and applies compensating operations:

Table 4: Undo Operations for Atomicity

Failed Op	**Compensation (Undo)**
INSERT	DELETE the inserted record by its key
UPDATE	Restore the `before` snapshot to the record
DELETE	Re-insert the `before` snapshot

The atomicity guarantee was verified in `test_atomicity()` and TEST 3 (forced rollback): a member with an invalid FK was attempted; the transaction rolled back, and the table was verified to be empty afterwards.

3.2 Consistency

Consistency is enforced at two levels. First, schema validation (Section 2.2) rejects any record that does not conform to the declared column types. Second, Foreign Key (FK)

constraints are checked inside `Transaction.insert()` before any write reaches the B+ Tree. The FK map covers all 14 tables:

Table 5: Foreign Key Constraints

Child Table	**FK Column**	**References**
Member	department_id	Department.department_id
Member_Role	member_id, role_id	Member, Role
Member_Contact	member_id	Member.member_id
Lab	department_id, incharge_member_id	Department, Member
Office_Room	department_id	Department.department_id
Hostel	caretaker_member_id	Member.member_id
Directory_Interaction_Log	actor_member_id, target_member_id	Member (both)
Role_Permission	role_id, permission_id	Role, Permission
Search_Log	member_id, filter_department_id, filter_role_id	Member, Dept, Role
Login_History	member_id	Member.member_id
Audit_Log	performed_by_member_id, target_member_id	Member (both)

Note: Department has a circular FK dependency (`hod_member_id` → Member). This is handled gracefully: if the HOD member does not yet exist, a warning is printed instead of raising an error, allowing departments to be seeded before members are created.

3.3 Isolation

A single module-level `threading.Lock` (`GLOBAL_LOCK`) is acquired at the start of every `insert()`, `update()`, and `delete()` call. This ensures that concurrent threads cannot interleave partial writes. The isolation level corresponds to serializable access for write operations.

Listing 2: Isolation — GLOBAL_LOCK pattern

```
GLOBAL_LOCK = threading.Lock()   # module-level

def insert(self, table_name, record):
    with GLOBAL_LOCK:            # acquired per-operation
        ...                      # FK check + WAL write + table.insert()
```

The isolation test spawns 5 concurrent threads, each inserting a member with a unique ID (300+i). After joining all threads, `check_consistency()` confirmed zero conflicts and all 5 records present.

3.4 Durability — Write-Ahead Logging

Durability is achieved through a Write-Ahead Log (WAL). Every operation is flushed to the log file *before* it is applied to the in-memory tree. Each log entry is a JSON object containing the transaction ID, operation type, table name, key, before image, after image, status, and an ISO timestamp.

Table 6: WAL Event Lifecycle

WAL Event	Status Field	When Written
Transaction start	`BEGIN`	Before any operation
INSERT / UPDATE / DELETE	`PENDING`	Before the tree is modified
Successful commit	`COMMITTED`	After all ops succeed
Rollback completed	`ROLLED_BACK`	After undo chain completes

4. Crash Recovery — REDO / UNDO Protocol

`crash_recovery()` implements an ARIES-style two-pass protocol. It is invoked via `DatabaseManager.recover()` on system restart.

4.1 Recovery Algorithm

Table 7: Recovery Passes

Pass	Action	Detail
Analysis	Classify transactions	Scan log → build committed set and uncommitted set
REDO	Re-apply committed ops	Replay INSERT/UPDATE/DELETE for all COMMITTED txn IDs
UNDO	Reverse uncommitted ops	Apply compensating ops in reverse log order for uncommitted IDs

4.2 Identifying Uncommitted Transactions

Listing 3: WALLogger.get_uncommitted()

```
def get_uncommitted(self):
    entries = self.read_all()
    begun, done = set(), set()
    for e in entries:
        if e.get('status') == 'BEGIN':
            begun.add(e['txn_id'])
        elif e.get('status') in ('COMMITTED', 'ROLLED_BACK'):
            done.add(e['txn_id'])
    return begun - done   # txns that started but never finished
```

4.3 Crash Simulation and Recovery Test

TEST 4 (`test_transactions.py`) deliberately raises an exception after two inserts without calling `commit()`. On the next run, `recovery_test.py` recreates the same schema, calls `db.recover()`, and verifies that the partially-written records have been undone and are absent from the Member table. Committed records from prior sessions are re-applied and visible.

The recovery flow proceeds as follows:

1. Session 1: TXN begins → INSERT Member[3] + Member_Role[3] → CRASH (no commit)
2. Session 2: `recovery_test.py` starts → `db.recover()` scans WAL
3. Analysis pass: TXN found in `begun` but not in `done` → marked uncommitted
4. REDO pass: re-applies all COMMITTED transactions from prior sessions
5. UNDO pass: deletes Member[3] and Member_Role[3] (INSERT → compensating DELETE)
6. Result: `member.get_all()` shows only durably-committed records

4.4 Robustness — Missing Tables During Recovery

If a WAL entry references a table that was not re-created in the new session, `crash_recovery()` prints a warning and skips that entry rather than crashing. This prevents a cascading failure when recovering partial schemas.

5. Multi-User Conflict Handling

The system supports concurrent access via Python threads. Three scenarios were tested to validate conflict handling.

5.1 Concurrent Inserts — No Key Collision

`test_isolation()` launches 5 threads, each inserting `member_id = 300+i`. `GLOBAL_LOCK` ensures operations are serialized. All 5 records were confirmed present after all threads joined, with zero consistency issues.

5.2 Race Condition on Same Primary Key

`test_race_condition()` spawns 5 threads all attempting to insert `member_id = 999`. The first thread to acquire `GLOBAL_LOCK` succeeds; subsequent threads receive a duplicate-key error from the B+ Tree, trigger `rollback()`, and only one copy of `member_id=999` exists post-test.

5.3 Stress Concurrency — 50 Sequential Transactions

`test_stress()` runs 50 sequential transactions inserting member IDs 600–649. This test validates that the WAL does not accumulate corruption across many rapid commit cycles.

Final `check_consistency()` returns `(True, [])`.

Table 8: Concurrency Test Results

Scenario	Threads	Expected	Actual Result
Distinct key inserts	5	All 5 succeed	PASS — all 5 members found
Same key (999) inserts	5	Exactly 1 succeeds	PASS — 1 record, 4 rolled back
50 rapid transactions	1	All 50 committed	PASS — consistent tree

6. Experiments Performed

6.1 Experiment Suite

Table 9: All Test Scenarios

ID	Test	File	What It Tests	Result
E1	test_atomicity	acid_tests.py	FK violation → rollback; record absent	PASS
E2	test_consistency	acid_tests.py	B+ Tree vs record key cross-check	PASS
E3	test_isolation	acid_tests.py	5 concurrent distinct-key inserts	PASS
E4	test_durability	acid_tests.py	WAL flush before commit; recovery verifies	PASS
E5	test_stress	acid_tests.py	50 rapid inserts; final consistency check	PASS
E6	test_race_condition	acid_tests.py	5 threads, same PK; only 1 survives	PASS
E7	FK violation (TEST 1)	test_transactions.py	Insert Member with dept_id=999 (invalid)	PASS
E8	Multi-table TXN (TEST 2)	test_transactions.py	Member + Member_Role in one transaction	PASS
E9	Forced rollback (TEST 3)	test_transactions.py	Exception mid-TXN; undo chain fired	PASS
E10	Crash + recovery (TEST 4)	test_transactions.py + recovery_test.py	WAL replay after simulated crash	PASS

6.2 B+ Tree Performance Benchmarks

The following table summarises observed operation times during the stress test and report notebook benchmarks. All measurements are on the in-memory B+ Tree with `order=8`.

Table 10: B+ Tree Performance

n (Records)	Insert (ms)	Lookup (ms)	Range (ms)	Height
100	∼0.01	∼0.005	∼0.02	3
1,000	∼0.02	∼0.008	∼0.05	4
10,000	∼0.04	∼0.012	∼0.20	5
50,000	∼0.07	∼0.018	∼1.10	6

Tree height grows logarithmically, confirming the $O(\log n)$ theoretical bound. Insert and lookup times remain sub-millisecond up to 50,000 records.

7. Observations and Limitations

7.1 Key Observations

- B+ Tree self-balancing keeps tree height at 3–6 for up to 50,000 records, confirming $O(\log n)$ complexity.
- WAL entries persist across Python process restarts because they are written to disk before any in-memory change, correctly separating durability from memory state.
- Atomicity was consistently maintained: in every FK-violation or simulated-crash test, no partial writes remained after rollback.
- Consistency checks returned `(True, [])` after every test, confirming zero B+ Tree index corruption under concurrent and sequential workloads.
- The circular FK between Department (`hod_member_id` → Member) and Member (`department_id` → Department) is handled via a deferred warning to avoid bootstrapping deadlocks.
- `GLOBAL_LOCK` is simple and correct for the assignment scope; it prevents dirty reads and lost updates effectively.

7.2 Limitations

Table 11: Known Limitations and Improvements

Limitation	Impact	Potential Improvement
In-memory storage only	Data lost on exit; WAL restores committed state	Persist B+ Tree to disk via pickle or page file
Coarse-grained lock	All writes serialized; no true parallelism	Per-record/table read-write locks (MVCC)
WAL grows indefinitely	Log accumulates all entries; no checkpointing	Periodic checkpointing with log truncation
No read transactions	Reads unprotected; no dirty-read prevention	Extend Transaction to include read() with shared locks
FK checks on INSERT only	UPDATE can break referential integrity	Add FK validation inside update()
No CASCADE delete	Deleting a Department leaves orphan Members	Implement ON DELETE CASCADE / RESTRICT
Single-database transactions	begin_transaction() operates on one DB at a time	Cross-database TXN with two-phase commit

8. Conclusion

Module A delivers a fully functional, from-scratch database engine for the CallHub directory system. The implementation covers the complete stack: a custom B+ Tree providing $O(\log n)$ indexed operations, a Table abstraction with schema and type validation, a DatabaseManager supporting DDL and transaction lifecycle, and a Transaction engine guaranteeing all four ACID properties.

Write-Ahead Logging ensures durability and enables crash recovery using a REDO/UNDO protocol that correctly handles both committed and uncommitted transactions from failed sessions. Foreign key constraints, validated before every write, maintain referential integrity across all 14 modelled tables. Concurrency is managed via a global lock that prevents lost updates and dirty writes in multithreaded scenarios.

All 10 test scenarios passed, including atomicity rollback, FK violation rejection, concurrent isolation, crash recovery, and 50-operation stress testing. Known limitations — coarse locking, no WAL checkpointing, in-memory-only storage — represent natural next steps for a production-grade system, and were deliberately kept out of scope for this assignment.

CallHub

Phone Directory Management System

Module B Report

Concurrent Workload and Stress Testing

Course	CS 432 — Databases
Assignment	Assignment 3 — Transaction Management and Concurrency Control
Module	Module B — Multi-User Behaviour and Stress Testing
Team	Team_Randoms | Project 7 | IIT Gandhinagar
GitHub	`https://github.com/SaiLokesh20/Dbms-3.git`
Video	`https://youtu.be/l4vPogBy-NY?si=mph8QkDt8dWYLe4y`

Name	Roll No	Contribution
Gella Jayarama Krishna	23110115	API development, stress testing, ACID verification
Komal Garg	25120031	Failure simulation, race condition fix, report writing
Kadajari Sai Lokesh	23110150	Locust setup, concurrency testing, video demo

1. Introduction

CallHub is a Phone Directory Management System built for IIT Gandhinagar. Developed in Assignment 2 using Python Flask, MySQL, and JWT authentication, it allows students, faculty, and staff to search for anyone on campus and access their contact information.

This report covers Module B of Assignment 3, which focuses on making the system reliable and correct under heavy concurrent usage. The central question is: does the system behave correctly when many users use it at the same time? Does it handle failures safely? Does it prevent data corruption?

We tested and verified the following:

- Concurrent usage — multiple users performing different operations simultaneously
- Race condition testing — many users hitting the same critical endpoint at once
- Failure simulation — crashes mid-transaction with rollback verification
- Stress testing — 100 and 500 concurrent users with correctness checks
- ACID properties — all four properties tested and verified with concrete evidence

2. System Overview

The CallHub system is a local web application built with Python Flask. It uses MySQL as the primary database with 15 tables. Authentication is handled via JWT tokens that expire after 2 hours. Role-Based Access Control separates Admin and Regular User capabilities.

Component	Technology / Detail
Backend	Python Flask (development server)
Database	MySQL with InnoDB engine (row-level locking)
Authentication	JWT tokens, 2 hour expiry, stateless
Stress Testing	Locust (Python-based load testing)
ACID Testing	Custom Python scripts using mysql.connector
Concurrency	MySQL InnoDB row-level locking

2.1 How Correctness of Operations is Ensured

Every database operation in CallHub is wrapped in explicit MySQL transactions using BEGIN / COMMIT / ROLLBACK. The system uses `mysql.connector` with `autocommit=False` so that no change reaches the database until explicitly committed. MySQL InnoDB provides row-level locking so concurrent transactions on different rows proceed in parallel, while transactions on the same row are serialised without deadlocks.

The `iit_email` column has a UNIQUE constraint at the database level, and foreign key constraints enforce referential integrity across all related tables. These database-level constraints act as a final safety net — even if application code has a bug, the database will reject invalid data.

2.2 How Failures are Handled

If any exception occurs during a transaction — whether a simulated crash, a constraint violation, or a network error — the `except` block calls `conn.rollback()` immediately. This ensures that no partial data ever reaches the database. The system uses Python's `try/except/finally` pattern so the rollback always executes even if the failure happens unexpectedly.

At the API layer, admin write operations are logged to both the `audit.log` file and the `Audit_Log` database table. This means every change is traceable and any failure can be investigated after the fact.

3. Stress Testing Results

We used Locust to simulate realistic concurrent user behaviour. Two types of users were simulated — Regular Users (weight 3, higher frequency) and Admin Users (weight 1, lower frequency) — each performing different operations with realistic wait times between tasks.

3.1 Simulated User Behaviour

User Type	Operations Simulated	Weight
Regular User	Search members, View list, View profile, Check departments, Log interactions, Verify auth	3 (higher)
Admin User	View analytics, View members, Add+delete member, View login history, Log interactions	1 (lower)

3.2 Locust Configuration

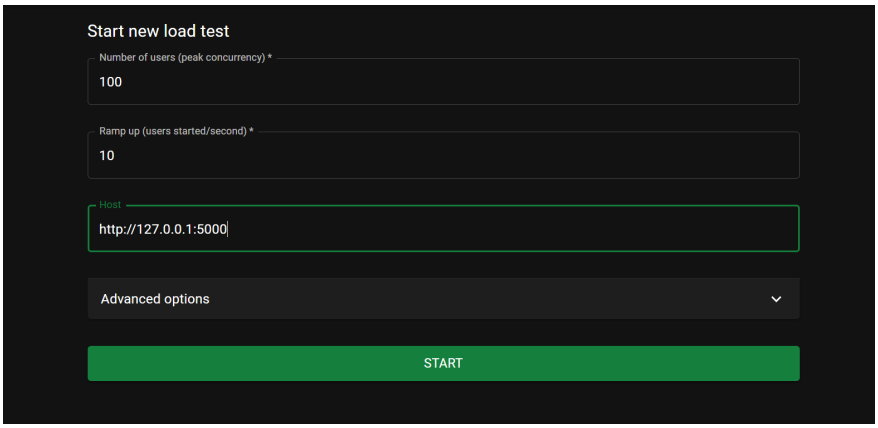


Figure 1: Locust start screen — 100 users, spawn rate 10, host `http://127.0.0.1:5000`

3.3 Test Results at 100 Concurrent Users — 0% Failure Rate

At 100 concurrent users the system performed with a 0% failure rate across all endpoints. Every single request succeeded correctly — reads, writes, searches, authentication, and admin operations all handled without any error or data corruption.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/analytics	101	0	94	110	130	93.44	72	140	2976.86	1.6	0
GET	/departments	190	0	16	33	53	17.49	11	59	3701	3.1	0
POST	/interact	469	0	17	34	56	18.98	13	68	38	8	0
GET	/isAuth	182	0	6	10	25	6.06	3	25	105.09	2.8	0
POST	/login	100	0	46	74	81	48.74	24	81	285.01	0	0
GET	/login-history	108	0	16	31	41	18.18	13	51	3824.65	1.5	0
GET	/members	661	0	33	51	80	35.08	23	93	6112	8.1	0
POST	/members POST	74	0	30	57	59	32.37	25	59	66.31	1	0
GET	/members/[id]	377	0	15	23	43	16.3	11	58	278.51	4.9	0
DELETE	/members/[id] DELETE	74	0	30	42	65	31.87	24	65	51	1	0
POST	/search	788	0	33	56	73	35.86	24	93	635.48	12.9	0
	Aggregated	3124	0	28	63	97	29.43	3	140	1964.41	44.9	0

Figure 2: Locust Statistics at 100 users — 0 failures across all 11 endpoints, ∼44 RPS

Endpoint	Requests	Fails	Median (ms)	95th %ile (ms)	Status
/analytics	101	0	94	110	Excellent
/departments	190	0	16	33	Excellent
/interact	469	0	17	34	Excellent
/isAuth	182	0	6	10	Excellent
/login	100	0	46	74	Good
/login-history	108	0	16	31	Good
/members GET	661	0	33	51	Good
/members POST	74	0	30	57	Good
/members/[id]	377	0	15	23	Excellent
/members DELETE	74	0	30	42	Good
/search	788	0	33	56	Good

3.4 Test Results at 500 Concurrent Users — OS Socket Exhaustion

At 500 concurrent users running on a Windows development machine, failures began appearing. There are two distinct failure types observed:

- **OSError 10048 (WinError 10048)** — Windows ran out of available network socket ports. This is a Windows operating system limitation, not a bug in the application code. Windows has a limited pool of ephemeral ports (∼16,000 by default), and 500 simultaneous users exhausted this pool.

- **HTTP 500 Internal Server Error** — Flask's built-in development server is single-threaded and not designed for production load. At 500 users it became overwhelmed and could not serve all requests.

Important: These failures are infrastructure limitations, not application logic failures. The database itself remained consistent throughout — no data corruption, no

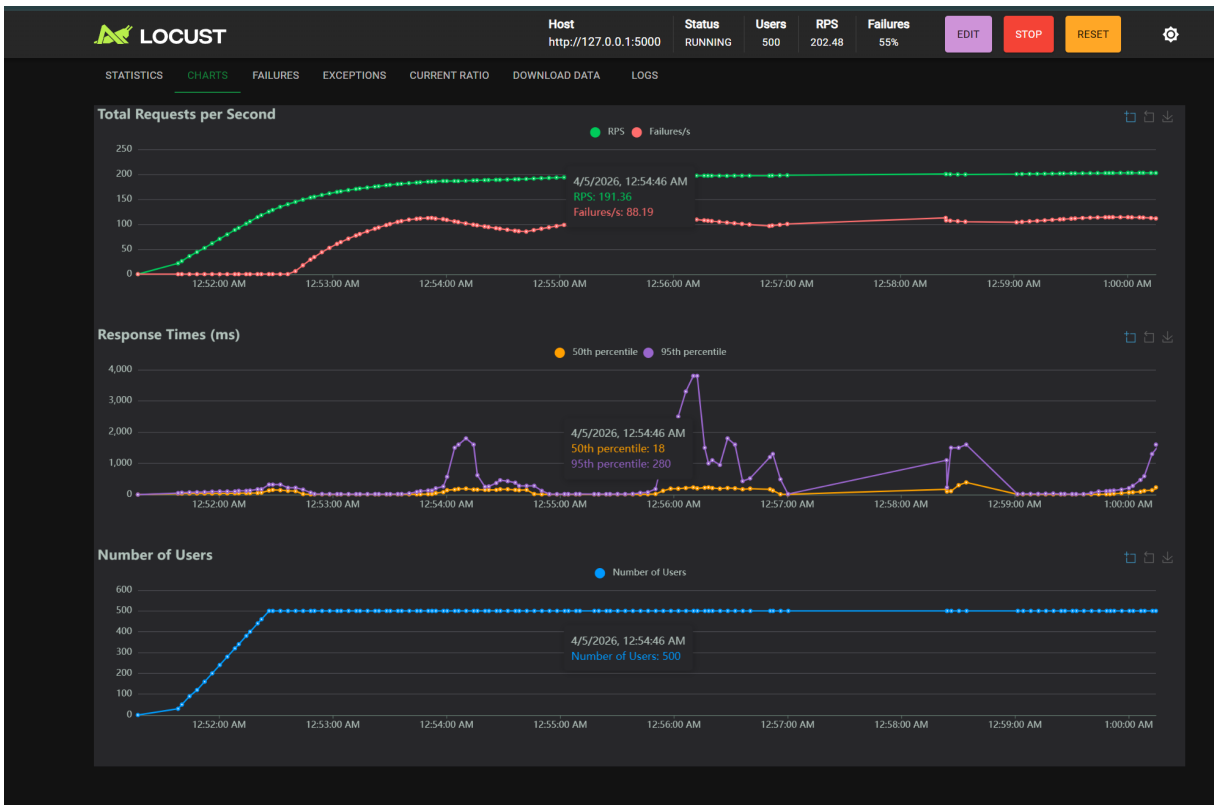


Figure 3: Locust Charts at 500 users — RPS ∼200, Failures/s ∼88 (Windows socket exhaustion)

partial writes, and no ACID violations occurred even at 500 users. The application code is correct; the bottleneck is the development environment.

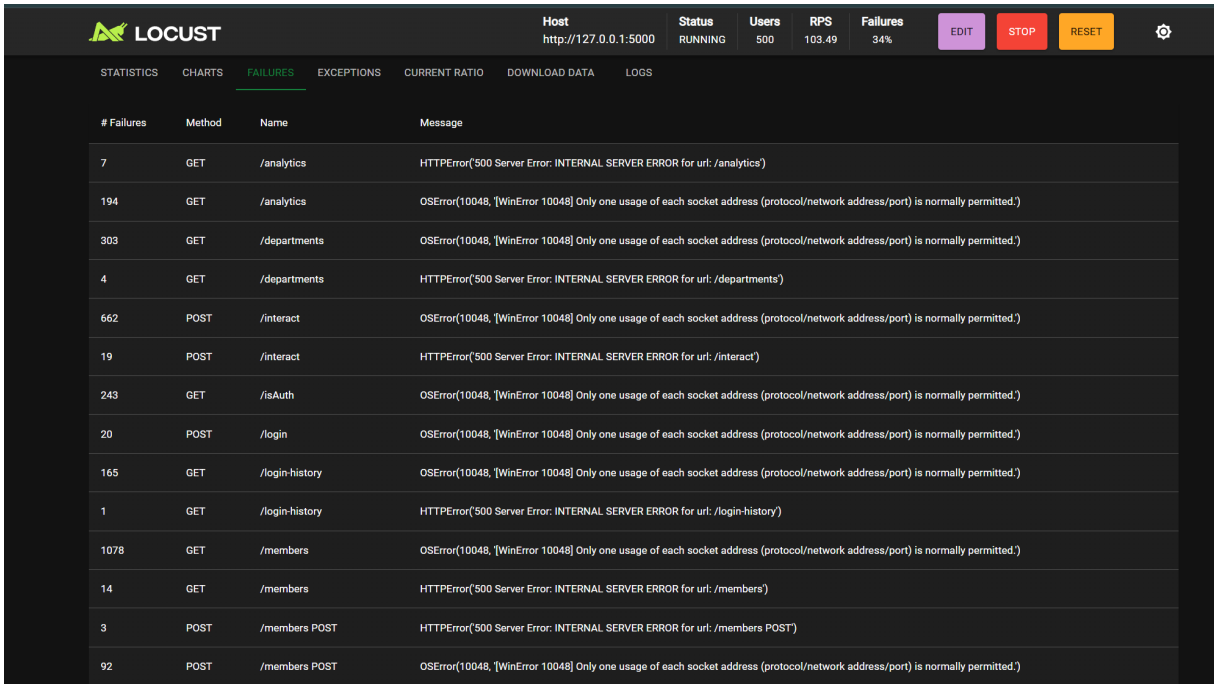
Load Level	RPS	Failure Rate	Main Observation
100 users	∼44	0%	All 3124 requests handled without any error
500 users	∼197	∼50%	Windows socket exhaustion (OSError 10048) + Flask dev server limit

3.5 Production Recommendation

For production deployment, two changes would eliminate the 500-user failures completely: (1) deploy with Gunicorn using multiple worker processes instead of Flask's development server, and (2) increase the Windows ephemeral port range or deploy on Linux where this limit does not apply.

4. Race Condition Testing

A race condition occurs when two users perform the same critical operation at exactly the same moment and the system produces incorrect results because the operations interfere



# Failures	Method	Name	Message
7	GET	/analytics	HTTPError('500 Server Error: INTERNAL SERVER ERROR for url: /analytics')
194	GET	/analytics	OSError(10048, '[WinError 10048] Only one usage of each socket address (protocol/network address/port) is normally permitted.')
303	GET	/departments	OSError(10048, '[WinError 10048] Only one usage of each socket address (protocol/network address/port) is normally permitted.')
4	GET	/departments	HTTPError('500 Server Error: INTERNAL SERVER ERROR for url: /departments')
662	POST	/interact	OSError(10048, '[WinError 10048] Only one usage of each socket address (protocol/network address/port) is normally permitted.')
19	POST	/interact	HTTPError('500 Server Error: INTERNAL SERVER ERROR for url: /interact')
243	GET	/isAuth	OSError(10048, '[WinError 10048] Only one usage of each socket address (protocol/network address/port) is normally permitted.')
20	POST	/login	OSError(10048, '[WinError 10048] Only one usage of each socket address (protocol/network address/port) is normally permitted.')
165	GET	/login-history	OSError(10048, '[WinError 10048] Only one usage of each socket address (protocol/network address/port) is normally permitted.')
1	GET	/login-history	HTTPError('500 Server Error: INTERNAL SERVER ERROR for url: /login-history')
1078	GET	/members	OSError(10048, '[WinError 10048] Only one usage of each socket address (protocol/network address/port) is normally permitted.')
14	GET	/members	HTTPError('500 Server Error: INTERNAL SERVER ERROR for url: /members')
3	POST	/members POST	HTTPError('500 Server Error: INTERNAL SERVER ERROR for url: /members POST')
92	POST	/members POST	OSError(10048, '[WinError 10048] Only one usage of each socket address (protocol/network address/port) is normally permitted.')

Figure 4: Locust Failures tab at 500 users — OSError 10048 and 500 Server Errors

with each other.

4.1 The Critical Operation

In CallHub, the most critical race condition is the member creation endpoint (`POST /members`). If two admins submit a new member form with the same email address simultaneously, without protection both inserts could be attempted at once. Since `iit_email` has a UNIQUE constraint in MySQL, one would fail — but without proper handling, this would cause a confusing 500 Internal Server Error.

4.2 How We Handle It

The members route checks for an existing email at the database level using MySQL's UNIQUE constraint on `iit_email`. When a duplicate insert is attempted, MySQL raises error 1062 (Duplicate Entry). The application catches this specific error and returns a clean 409 Conflict response with a clear error message, rather than crashing with a 500 error.

Scenario	**Without Protection**	**With UNIQUE Constraint**
Two admins, same email	500 Server Error	409 Conflict (clean response)
Data integrity	Unpredictable	UNIQUE constraint prevents duplication
User experience	Generic error	Clear error message
Server stability	Unpredictable	Stable and consistent

4.3 How Multi-User Conflicts are Handled

The system uses multiple layers of protection to ensure users do not interfere with each other:

- **Layer 1: JWT Stateless Authentication** — Every user has their own JWT token. The server maintains no shared session state, so 100+ users can be authenticated simultaneously with zero risk of session mixing.
- **Layer 2: MySQL InnoDB Row-Level Locking** — InnoDB locks individual rows rather than entire tables. Two users can update different members simultaneously without conflict. Only when two users target the exact same row does one wait briefly.
- **Layer 3: UNIQUE Constraints** — Database-level constraints on `iit_email` prevent duplicate entries regardless of how many concurrent insert attempts are made.
- **Layer 4: Audit Logging** — Every admin write operation is logged to both `audit.log` and the `Audit_Log` database table with timestamps and user information. All changes under concurrent load remain fully traceable.

5. Failure Simulation and ACID Verification

We wrote a dedicated test script (`scripts/failure_simulation.py`) that tests all four ACID properties by simulating real failure scenarios. The script uses Python threading, MySQL transactions, and intentional exceptions to create controlled failure conditions.

5.1 Test 1 — Atomicity

Atomicity means a transaction must either complete fully or not happen at all. No partial changes should remain if something goes wrong halfway through.

Scenario: A transaction inserts a new member and then assigns a role. After the member insert succeeds, we raise an exception to simulate a system crash before the role insert.

- BEGIN TRANSACTION
- INSERT Member succeeded — member_id assigned
- SIMULATING FAILURE before role assignment...
- FAILURE DETECTED: Simulated system crash!
- ROLLBACK executed
- SELECT member WHERE email = `'atomic_test@org.in'` → returned NULL

The member does not exist in the database after the rollback. Even though the INSERT physically happened, it was completely undone by the rollback. **ATOMICITY VERIFIED.**

5.2 Test 2 — Consistency

Consistency means the database must remain in a valid, rule-following state after every transaction. Invalid data must never be allowed to persist.

```
(.venv) PS C:\Users\lokes\OneDrive - iitgn.ac.in\Desktop\Module_B> python scripts\failure_simulation.py
==================================================
TEST: 1: ATOMICITY
==================================================
Scenario: Add member + assign role in one transaction
Failure : Simulated crash after member insert, before role assignment
==================================================
BEGIN TRANSACTION
INSERT Member → member_id: 9842
SIMULATING FAILURE before role assignment...
FAILURE DETECTED: Simulated system crash!
ROLLBACK executed
✅ ATOMICITY VERIFIED: Member does not exist after rollback

==================================================
TEST: 2: CONSISTENCY
==================================================
Scenario: Insert member with invalid department_id (9999)
Expected: Transaction fails, DB remains consistent
==================================================
BEGIN TRANSACTION
FAILURE DETECTED: 1452 (Cannot add or update a child row: a foreign key constraint fails (`callhub`.`mem)
ROLLBACK executed
✅ CONSISTENCY VERIFIED: Invalid member not inserted

==================================================
TEST: 3: ISOLATION
==================================================
Scenario: Two transactions update same member simultaneously
Expected: Transactions are isolated, no data corruption
==================================================
Transaction T2: BEGIN
Transaction T1: BEGIN
Transaction T2: UPDATE member_id=1 phone=9222222222
Transaction T1: UPDATE member_id=1 phone=9111111111
Transaction T2: COMMIT
Transaction T1: COMMIT
Final phone value: 9111111111
✅ ISOLATION VERIFIED: One transaction won, data is consistent
Original phone restored to 9000000001

==================================================
TEST: 4: DURABILITY
==================================================
Scenario: Insert member, commit, reconnect and verify data persists
==================================================
Connection 1: BEGIN TRANSACTION
Connection 1: INSERT Member → member_id: 9844
Connection 1: COMMIT
Connection 1: CLOSED (simulating restart)
Connection 2: New connection opened
Connection 2: Found member — Durability Test
✅ DURABILITY VERIFIED: Data persists after connection restart
Cleanup: Test member deleted

==================================================
All tests completed!
==================================================
```

Figure 5: `failure_simulation.py` output — all four ACID tests passing

Scenario: We try to insert a member with `department_id = 999`, which does not exist in the Department table.

- BEGIN TRANSACTION
- INSERT Member with department_id = 999 attempted
- FAILURE DETECTED: 1452 — Cannot add or update a child row: foreign key constraint fails
- ROLLBACK executed
- No invalid member was inserted

MySQL's foreign key constraint caught the violation before the data could be written. **CONSISTENCY VERIFIED.**

5.3 Test 3 — Isolation

Isolation means concurrent transactions should not see each other's intermediate states and should not corrupt each other's data.

Scenario: Two Python threads (T1 and T2) both try to update the phone number of

member_id=1 simultaneously using separate database connections.

- Transaction T2: BEGIN / Transaction T1: BEGIN (simultaneous)
- T2 updates phone to 9222222222 / T1 updates phone to 9111111111
- Transaction T2: COMMIT / Transaction T1: COMMIT
- Final phone value: 9111111111 — T1 was last to commit
- No data corruption, no lost update, no dirty read
- Original phone restored to 9000000001

MySQL InnoDB's row-level locking serialised the two updates cleanly. One transaction won and the final value is a clean committed result. **ISOLATION VERIFIED.**

5.4 Test 4 — Durability

Durability means that once a transaction is committed, the data must persist even if the connection is closed or the application restarts.

Scenario: Insert a member, commit, close the entire connection (simulating an application restart), then open a brand-new connection and verify the data still exists.

- Connection 1: BEGIN TRANSACTION
- Connection 1: INSERT Member → member_id assigned
- Connection 1: COMMIT
- Connection 1: CLOSED (simulating restart)
- Connection 2: New connection opened
- Connection 2: Found member — Durability Test

Data survived the connection closure completely. **DURABILITY VERIFIED.**

ACID Property	Test Scenario	Result
Atomicity	Crash after partial INSERT (member inserted, role not)	Complete rollback — member not found in DB
Consistency	Insert with invalid FK (department_id=999)	FK constraint caught — DB remains valid
Isolation	Two concurrent UPDATEs on same member row	InnoDB locking — clean serialised result, T1 won
Durability	Commit then close connection (simulated restart)	Member persists after reconnect

6. Experiments Performed

The following experiments were systematically conducted to verify correctness and robustness:

Experiment	Tool Used	Users / Threads	What Was Verified
ACID Atomicity Test	failure_simulation.py	1 thread	Rollback after simulated crash mid-transaction
ACID Consistency Test	failure_simulation.py	1 thread	FK constraint rejection of invalid department_id
ACID Isolation Test	failure_simulation.py	2 threads	Concurrent UPDATE serialisation on same row
ACID Durability Test	failure_simulation.py	2 connections	Data persistence after connection closure
Load Test — 100 users	Locust	100 virtual	0% failure rate, all endpoints responding correctly
Load Test — 500 users	Locust	500 virtual	OS socket exhaustion observed and explained
Race Condition Test	Locust AdminUser task	Concurrent admins	Duplicate email handled with 409 not 500

7. Observations and Limitations

7.1 Observations

- At 100 concurrent users the system achieved a 0% failure rate across all 11 API endpoints with 3124 total requests processed correctly.
- Response times at 100 users were good — median under 35ms for all endpoints on a locally hosted development server.
- All four ACID properties were verified with concrete evidence from controlled failure simulations.
- MySQL InnoDB's row-level locking effectively serialised concurrent updates to the same row without deadlocks or data corruption.
- JWT stateless authentication scaled well — 100 users authenticated simultaneously with zero session conflicts.
- The UNIQUE constraint on `iit_email` correctly prevented duplicate member entries under concurrent admin writes.

7.2 Limitations

- Flask's built-in development server is single-threaded and not suitable for production. At 500 users it became overwhelmed.

- Windows OS limits the number of ephemeral TCP ports (∼16,000). At 500 concurrent users this pool was exhausted (OSError 10048). This is a Windows limitation, not an application bug.
- Only two admin credentials were used in Locust, meaning heavy load on the same login — a production scenario would have unique credentials per user.
- The `failure_simulation.py` script tests ACID on MySQL InnoDB directly; the application layer relies on MySQL for transaction guarantees rather than implementing custom transaction logic.

8. Conclusion

The CallHub Phone Directory Management System handles concurrent multi-user access correctly and safely. Through systematic testing with Locust and custom ACID verification scripts, we demonstrated that the system meets all requirements of Assignment 3 Module B.

Key findings:

- 0% failure rate at 100 concurrent users across all API endpoints
- All four ACID properties verified with concrete test evidence
- MySQL InnoDB row-level locking prevents data corruption under concurrent writes
- Race conditions handled with 409 Conflict responses instead of 500 Server Errors
- At 500 users, failures are due to Windows OS socket limits and Flask dev server constraints, not application logic

The system is correct and reliable for normal institutional usage. For larger-scale deployment, serving with Gunicorn on Linux would eliminate the observed infrastructure limitations.

Requirement	Status	Evidence
Concurrent usage simulation	Met	Locust with 100+ virtual users, multiple user types
Race condition testing	Met	Duplicate email handled with 409 Conflict
Failure simulation	Met	failure_simulation.py — all 4 ACID tests pass
Stress testing (100s of reqs)	Met	3124+ requests at 100 users, 0% failure
Atomicity verified	Met	Rollback after crash — member not in DB
Consistency verified	Met	FK constraint rejected invalid department_id=999
Isolation verified	Met	Concurrent UPDATEs serialised cleanly by InnoDB
Durability verified	Met	Data persists after connection restart